Data-Aware Task Scheduling for Approximate DAG Workflows in Federated Fog Systems

Gayani Gupta*, Amisha Gupta†, Mohsen Amini Salehi*,
Antonio Fernández Anta‡,§, Sanjukta Bhowmick*, José Aguilar§,¶
*University of North Texas, USA †UC Berkeley, USA ‡IMDEA Networks Institute, Spain
§IMDEA Software Institute, Spain ¶Universidad de Los Andes, Venezuela gayani.gupta@unt.edu

Abstract—Flood forecasting, wildfire tracking, and evacuation planning all run as workflows of dependent tasks. During an extreme event, the cloud link often fails, so the work must be done on small fog computers near the disaster area. A single fog site rarely has enough capacity to finish in time, but nearby sites can still reach each other through fast links, so a federation of them is the natural place to run the workflow. Scheduling on a federation is difficult for two reasons. First, a task's input data can take as long to arrive as the task takes to run. Second, many tasks can be run approximately: they give a less accurate result in less time, which is often the right trade-off under a deadline.

We write this scheduling problem as a mixed-integer linear program (MILP) that decides which fog node runs each task and how much of it runs, maximizing the workflow's average accuracy subject to its deadline. The model takes per-edge data sizes and per-link bandwidths as inputs; to our knowledge, no earlier model treats these inputs and these two decisions together. Since the MILP is only practical at moderate sizes, we also build three faster schedulers on the same cost model—linear-program (LP) relaxation with randomized rounding, a bandwidth-aware greedy rule, and a genetic algorithm—and compare all four in simulation. Under tight deadlines the MILP wins by a clear margin; with generous slack the MILP and the greedy both reach the accuracy bound; and at the largest sizes the simple greedy is nearly as good while running far faster.

Index Terms—Data-aware scheduling, federated fog systems, directed acyclic graphs, approximate computing, mixed-integer linear programming, genetic algorithms.

I. Introduction

Industrial and emergency applications increasingly run on fog computers: small compute sites close to the sensors, deployed where the data is produced. This is especially important in remote or extreme environments like offshore platforms, deep-sea facilities, or disaster zones, where the link to a cloud data center is slow, intermittent, or simply absent. When the cloud cannot be reached, a local fog site must handle time-critical work on its own, and a single site often does not have enough capacity to finish a workflow before its deadline.

A practical way to add capacity is to federate nearby fog sites. During a wildfire, for example, the computers at an evacuation shelter, a field hospital, and a mobile command unit can still reach each other over local radio links even when the wide-area network is down. A federated fog then works like a small local cloud: tasks can be placed on any node, and nodes share data over the inter-fog links [1], [2].

Federation, however, makes scheduling harder, not easier. Real-time industrial applications are usually written as directed acyclic graphs (DAGs), where an edge means that one task must receive another task's output before it can start. Those edges carry real data, such as sensor streams, simulation checkpoints, or model features. Moving that data between nodes takes time that depends on the link bandwidth. Most existing fog schedulers reason only about compute: they ask where a task runs fastest, but not how long it takes to get the task's input there. In a disaster-response workflow, this omission is costly because a "fast" remote placement can lose more time to data transfer than it saves in execution.

There is a second factor that data-unaware schedulers also ignore: tasks can run approximately. Many analytics tasks produce a less accurate result in less time: sampling a simulation fewer times, truncating a feature pipeline, or stopping an iterative method early. When a deadline is tight, running a task at 80% accuracy may be the difference between a warning that arrives in time and one that arrives too late.

This paper integrates both aspects into one scheduling model. The contributions are:

- A data- and bandwidth-aware formulation of approximate DAG scheduling in federated fog systems as a mixed-integer linear program (MILP, Section III): maximize average accuracy under precedence, communication delay, placement eligibility, and one workflow deadline. To our knowledge, per-edge data size, per-link bandwidth, and per-task execution fraction have not previously been modeled jointly in this setting.
- Three scalable schedulers on the same cost model: LP relaxation with randomized rounding, a bandwidth/data-aware greedy algorithm, and a genetic algorithm (Section IV).
- A simulation study varying deadline slack, fog-network size, workflow size, task failure rate, and link bandwidth (Section V).
- A correctness argument and a linearization suitable for standard MILP solvers (the Appendix).

An earlier version of the scheduling model appeared in a conference paper [3]; this work adds the complete

data- and bandwidth-aware formulation with proofs and linearization, the failure-rate and bandwidth-degradation experiments, and the comparison against a bandwidth-blind baseline.

II. Related Work

Scheduling a DAG of tasks on heterogeneous processors is a well-studied problem. The standard reference heuristic is HEFT [4], which ranks each task by the remaining work plus communication on the longest path to the end of the graph and places each task, in rank order, on the processor where it finishes earliest. Our model uses the same convention that the transfer cost lives on the DAG edge. The difference is what a task is allowed to be: HEFT assumes a task runs completely or not at all, and so do the DAG schedulers used in mobile edge computing [5] and so do recent learned schedulers. In our model a task can run partially; the execution fraction l_{ia} is a decision variable, so a workflow that cannot finish at full quality can still finish with less accuracy. Partitioning a task graph for data-intensive workflows has been studied before [6], but without per-link bandwidth or partial execution.

Fog and edge resource allocation is surveyed in [2], and federating nearby fog sites has been proposed for industrial settings [1]. Inside such a federation, existing schedulers mostly worry about compute: where to place components for throughput [7] or quality-of-service targets. What they largely ignore is the data that must travel between dependent tasks. In a federated fog the links between sites are the scarce resource, so a scheduler that ignores them can pick a plan that looks fast on paper but misses its deadline in the field. Our formulation avoids that by putting the data volume on every edge and the bandwidth on every link directly into the optimization, together with partial execution.

Trading result quality for resources is not new: probabilistic task pruning drops low-value tasks to keep a schedule feasible under load [8]. We use a softer version: instead of dropping a task, the scheduler can run part of it. That difference matters in disaster response, where dropping the alerting task is not an option but running a faster, slightly less accurate version of it may be.

III. System Model and Problem Statement

A. Setting

We consider a federation of fog nodes $F = \{f_1, \dots, f_m\}$ deployed across a wildfire-prone region—a gateway at the field station, a shelter, a hospital, and a mobile response unit (Fig. 1). Each pair of nodes is connected by a link of known bandwidth, and the federation keeps working when the link to the cloud is down, which during a fire is the common case rather than the exception. A workflow arrives as a DAG $G = (T, E)$, where each node $T_i \in T$ is a computational task, and each edge $(T_j, T_i) \in E$ is both a precedence constraint and a data transfer of size D_{ji}.

TABLE I: Parameters and decision variables.

Symbol	Meaning
T_i	task i of the workflow DAG
f_a	fog node a
t_{ia}	full execution time of T_i on f_a
A_i	accuracy delivered if T_i runs completely
D_{ji}	data transferred from T_j to T_i
B_{ba}	bandwidth of link (f_b, f_a)
P_{ia}	1 if T_i may run on f_a, else 0
in_i	predecessors of T_i in the DAG
$t_{\max}$	workflow deadline
e_{ia}	1 if T_i is assigned to f_a
l_{ia}	fraction of T_i executed on f_a
M_{ia}	earliest time T_i can start on f_a
y_i	completion time of T_i

If task T_j runs on node f_b and its successor T_i runs on node f_a, the input of T_i has to cross the link (f_b, f_a) with bandwidth B_{ba}. That transfer takes:

$$c_{ji}^{ba} = \begin{cases} 0, & f_b = f_a, \\ \dfrac{D_{ji}}{B_{ba}}, & f_b \neq f_a. \end{cases} \tag{1}$$

This is the central cost in the model: placing a task is not only about how fast a node computes, but also about how long its input data take to arrive.

B. Assumptions

Tasks are non-preemptive, each task runs on at most one node, inputs from all predecessors can arrive in parallel when the links allow it, and concurrent tasks on the same node do not slow each other down. The important assumption is the last one: a task can run approximately, and executing fraction l of it costs fraction l of its full time and earns fraction l of its accuracy. Running T_i at l_{ia}=0.5 produces half of a full-quality result—an iterative solver stopped after half its iterations, a video pipeline processing every other frame. This linear trade-off is a first-order model, not a law, but it makes the deadline–accuracy trade-off explicit and keeps (2)–(10) linear.

C. The Scheduling Problem

Table I lists the notation. Each task T_i has a full-accuracy value A_i, an execution time t_{ia} on each node f_a, and a placement-eligibility indicator P_{ia}. The workflow must finish by a deadline $t_{\max}$.

The problem is then to choose, for every task, which node runs it and how much of it runs, so that the average delivered accuracy is as high as possible and that every task finishes by $t_{\max}$.

D. MILP Formulation

The objective is to maximize the average accuracy over the workflow:

$$\max \quad \frac{1}{n} \sum_{T_i \in T} A_i \sum_{f_a \in F} l_{ia}. \tag{2}$$

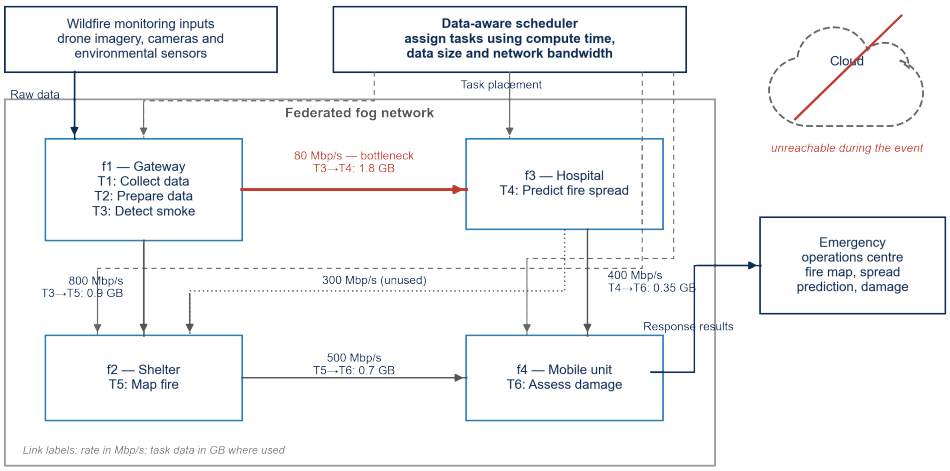


Fig. 1: Federated fog deployment for wildfire response. Smoke cameras, drones, and weather stations feed a workflow of dependent tasks; the scheduler assigns each task to one fog node (f_1 Gateway, f_2 Shelter, f_3 Hospital, f_4 Mobile — the same nodes as Fig. 2), priced by both compute time and inter-node data-transfer cost.

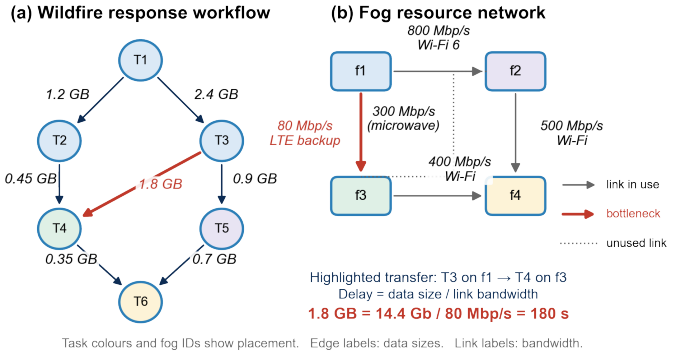


Fig. 2: A wildfire-response workflow (a) mapped onto a federated fog network (b): imagery is ingested, environmental data prepared, smoke detected, the fire spread predicted and mapped, and damage assessed. Edge labels are data sizes D_{ji}; link labels are bandwidths B_{ba}; the fog node inside each task shows its placement e_{ia}. The highlighted transfer—smoke-detection frames T_3 on f_1 feeding spread prediction T_4 on f_3—pays $D_{ji}/B_{ba} = 14.4\,\text{Gb}/80\,\text{Mbp/s} = 180\,\text{s}$ (1.8 GB expressed in gigabits), the transfer delay of Eq. (1).

subject to

$$\sum_{f_a \in F} e_{ia} = 1, \qquad \forall T_i \in T, \tag{3}$$

$$e_{ia} \le P_{ia}, \qquad \forall T_i,\ f_a, \tag{4}$$

$$0 \le l_{ia} \le e_{ia}, \qquad \forall T_i,\ f_a, \tag{5}$$

$$M_{ia} \ge y_j + \sum_{f_b \in F} e_{jb}\, c^{ba}_{ji}, \qquad \forall f_a,\ \forall T_i,\ \forall T_j \in \text{in}_i, \tag{6}$$

$$M_{ia} = 0, \qquad \forall f_a,\ T_i : \text{in}_i = \varnothing, \tag{7}$$

$$y_i = \sum_{f_a \in F} e_{ia}\big(M_{ia} + t_{ia} l_{ia}\big), \qquad \forall T_i, \tag{8}$$

$$y_i \le t_{\max}, \qquad \forall T_i, \tag{9}$$

$$e_{ia} \in \{0,1\}. \tag{10}$$

Constraints (3)–(4) place every task on exactly one permitted node. Constraint (5) ties the execution fraction to the assignment: a task can only run where it is placed, and at most one l_{ia} per task is non-zero. The case $l_{ia} = 0$ means the task is placed but never runs; it earns no accuracy, but it still hands its output data D_{ji} to every successor, so the transfer cost in (1) is charged whether or not the task ran. Skipping is rare in practice: across our runs it occurs only in time-limited MILP solves, never in a certified optimum. Constraint (6) is the data-aware term: a task cannot start before every predecessor has finished and its output has crossed the link. Equation (8) defines the completion time and (9) enforces the deadline. The products $e_{ia}M_{ia}$ in (8) are quadratic; the Appendix gives the standard linearization ($z_{ia} = e_{ia}M_{ia}$) and the correctness argument.

Scheduling problems of this kind are NP-hard already on unrelated parallel machines, so we treat the MILP as the benchmark: it gives the best achievable accuracy on instances small enough to solve exactly and defines the constraints every scalable method must respect. For larger workflows we use three cheaper methods: randomized rounding, a greedy scheduler, and a genetic algorithm.

IV. Scalable Scheduling Algorithms

All three methods take the same inputs as the MILP: the DAG, the times t_{ia}, the data sizes D_{ji}, the bandwidths B_{ba}, the eligibility flags P_{ia}, and the deadline $t_{\max}$. Because they all evaluate the communication term (1), they are all bandwidth-aware.

A. LP Relaxation with Randomized Rounding

The first method relaxes the binary variables of Section III-D to $e_{ia} \in [0,1]$ and solves the resulting LP—namely, the linearized formulation of the Appendix, since

(8) contains the product $e_{ia}M_{ia}$ and is only linear after the z_{ia} substitution. Let r_{ia} be the value the LP gives to e_{ia}; we can read it roughly as the probability that node f_a is a good place for T_i. We then assign each task to a node by sampling this distribution, renormalized over the eligible nodes $\{a : P_{ia} = 1\}$. Once the assignment is fixed, a second LP solve sets the execution fractions l_{ia} and the timing values.

The rounding inherits bandwidth awareness from the LP itself: a placement that needs a large D_{ji} over a slow link gets a small r_{ia} and is rarely sampled. Infeasible samples are discarded and re-sampled. The method sits between the MILP and the heuristics: no optimality guarantee, but a distribution from an actual optimization rather than a rule of thumb.

B. Bandwidth- and Data-Aware Greedy Scheduler

The greedy scheduler processes the DAG in dependency order and assigns each ready task to the node that finishes it earliest. Among the ready tasks, it picks the one with the highest priority, which mixes accuracy with urgency:

$$\pi_i \;=\; \beta \cdot A_i \;+\; (1-\beta)\cdot \mathrm{CP}_i, \tag{11}$$

where CP_i is the critical-path length from T_i to a sink under the current t_{ia} and c^{ba}_{ji}, and $\beta \in [0,1]$ trades accuracy importance against schedule urgency (default $\beta = 0.5$; ties break by accuracy density, i.e. A_i divided by the task's mean eligible execution time). Since A_i and CP_i have different units, (11) is a heuristic priority score rather than a physical quantity: A_i is an order-one accuracy weight while CP_i is measured in time units, so β controls the trade-off at the relative scale of the two ranges rather than in absolute terms.

The scheduler keeps a single global execution fraction $\ell \in [\ell_{\min}, 1]$, applied to every task, and decreases it by a step δ whenever the produced schedule misses the deadline, down to a floor $\ell_{\min} > 0$ at which the run is declared infeasible:

Complexity: One pass costs $O(nm+m|E|)$—each task scans all m nodes and maximizes over its in-neighbours—and the number of passes is bounded by $\lceil (1-\ell_{\min})/\delta \rceil$.

The greedy is a practical method rather than just a baseline: it accounts for the deadline through ℓ, bandwidth through c^{ba}_{ji}, and node speed through t_{ia}. What it cannot do is look ahead—a cheap early decision can force an expensive one later—which the genetic algorithm addresses.

We also tried a variant that keeps a separate fraction per task. After each pass it finds the chain of tasks that sets the finish time and cuts only those; the rest keep their full accuracy. Cutting the whole workflow wastes work on tasks that were never in the way, and the difference is large: at α=0.5 this version reaches 0.74 of the bound at n=100 against 0.41 for the uniform one, while still meeting every deadline. Once the deadline is loose ($\alpha \geq 1.2$) the two are identical.

Algorithm 1 Bandwidth/Data-Aware Greedy DAG Scheduler

$\ell \leftarrow 1$; compute priorities π_i by (11)
repeat
  $y_i \leftarrow \infty$ for all T_i; ready set $\mathcal{R} \leftarrow$ source tasks
  while $\mathcal{R} \neq \varnothing$ do
    pick $T_i \in \mathcal{R}$ with maximum π_i
    $y^\star \leftarrow \infty$; $a^\star \leftarrow \perp$
    for each $f_a \in F$ with $P_{ia} = 1$ do
      $M_{ia} \leftarrow \max_{T_j \in \mathrm{in}_i} \left(y_j + \sum_{f_b \in F} e_{jb}\, c^{ba}_{ji}\right)$
      $y_i^{(a)} \leftarrow M_{ia} + t_{ia}\,\ell$
      if $y_i^{(a)} < y^\star$ then
        $y^\star \leftarrow y_i^{(a)}$; $a^\star \leftarrow a$
      end if
    end for
    $e_{ia^\star} \leftarrow 1$; $y_i \leftarrow y^\star$; release newly-ready successors into $\mathcal{R}$
  end while
  $Y_{\max} \leftarrow \max_i y_i$
  if $Y_{\max} > t_{\max}$ then
    $\ell \leftarrow \ell - \delta$ {e.g. $\delta = 0.005$}
  end if
until $Y_{\max} \leq t_{\max}$ or $\ell < \ell_{\min}$
if $Y_{\max} > t_{\max}$ then
  return infeasible {even $\ell = \ell_{\min}$ misses the deadline}
end if

C. Genetic-Algorithm Scheduler

A chromosome encodes a full placement plus a per-task execution fraction: an assignment vector $\mathbf{f}$ with $f(i) \in \{a : P_{ia} = 1\}$, and a fraction vector $\boldsymbol{\ell} \in [0,1]^n$ that maps to $l_{i,f(i)}$. Fitness is evaluated by a forward pass over the DAG,

$$M_{i,f(i)} = \max_{T_j \in \mathrm{in}_i} \left(y_j + c_{ji}^{f(j),f(i)}\right), \qquad y_i = M_{i,f(i)} + t_{i,f(i)}\ell_i, \tag{12}$$

and the fitness to maximize is

$$\Phi = \frac{1}{n}\sum_i A_i\,\ell_i - \lambda_1\,\mathbb{I}[Y_{\max} > t_{\max}] - \lambda_2\,\frac{(Y_{\max} - t_{\max})_+}{t_{\max}}, \tag{13}$$

where $Y_{\max} = \max_i y_i$, $\mathbb{I}[\cdot]$ is the indicator function, and $\lambda_1 \gg \lambda_2$ penalize deadline violation. Infeasible individuals ($P_{i,f(i)} = 0$) receive $\Phi = -\infty$.

Because the GA evaluates complete assignments instead of committing tasks one at a time, it can find structures the greedy misses, such as grouping a data-heavy subgraph on one well-connected node.

V. Performance Evaluation

A. Experimental Setup

All experiments use a custom discrete-event simulator. Workloads are synthetic disaster-response DAGs serialized in JSON, each task carrying an execution-time vector t_{ia}, edge data sizes D_{ji}, a full-accuracy value A_i, and eligibility

Algorithm 2 Genetic Algorithm for Bandwidth/Data-Aware Fog Scheduling

initialize population with feasible $(\mathbf{f}, \boldsymbol{\ell})$; seed co-location of predecessors joined by large D_{ji}
repeat
  evaluate Φ of each individual via (13)
  select parents (tournament or roulette)
  one-point crossover on $\mathbf{f}$ and $\boldsymbol{\ell}$ independently
  mutate: reassign $f(i)$ within $\{a : P_{ia} = 1\}$; jitter $\ell_i \leftarrow \mathrm{clip}_{[0,1]}(\ell_i + \mathcal{N}(0, \sigma^2))$
  repair: bias toward co-locating T_i with a predecessor T_j when D_{ji} is large and the link is slow
  elitism: keep top-k individuals
until maximum generations reached or no improvement

P_{ia}. The fog federation is generated with heterogeneous node speeds and link bandwidths, and communication delays follow Eq. (1). The parameters are chosen so that data transfers genuinely contest compute rather than being negligible or prohibitive: task times $t_{ia} \in [2, 6]$, edge data sizes $D_{ji} \in [10, 80]$, and link bandwidths $B_{ba} \in [10, 80]$. Reading one time unit as 100 s, one data unit as 100 MB, and one bandwidth unit as 8 Mbp/s, these correspond to task runtimes of roughly 3–10 minutes (analytics and inference steps on Jetson-class fog hardware), edge payloads of 1–8 GB (drone imagery bundles and weather rasters), and link rates of 80–640 Mbp/s (the Wi-Fi/LTE backhaul range shown in Fig. 2), which keeps D_{ji}/B_{ba} comparable to task execution time. Node speeds vary within $[0.8, 1.25]\times$—a modest band, so that outcomes are decided by data movement rather than by which node happens to be fastest—and roughly 15% of task–node pairs are ineligible, modeling missing sensor feeds or software dependencies.

We vary five factors. Test datasets are four seeded instance sets: 10 tasks/3 nodes, 100 tasks/5 nodes, 500 tasks/8 nodes, and 1000 tasks/10 nodes (the two large sets use a sparser DAG, density 0.02 vs. 0.15, so the edge count stays realistic; larger structures up to 10^6 tasks are generated in the companion notebook but do not fit the dense schedulable-instance path). Federation size runs $N \in \{2, 4, 6, 8, 10\}$. Deadline slack sets $t_{\max} = \alpha \cdot T_{\mathrm{cp}}$, with T_{cp} the critical-path length at full execution and $\alpha \in \{1.0, 1.2, 1.3, 1.5, 2.0\}$. Task failures apply a per-task rate from 0% to 50% under a retry-once model: a failed task runs once more in place, costing twice its time but keeping its accuracy if the retry still meets the deadline. Link bandwidth scales every inter-node bandwidth by a factor in $\{1.0, 0.5, 0.2, 0.1, 0.05\}$ with the deadline unchanged, modeling link degradation during an event. The methods compared are the MILP, randomized rounding, the greedy, and the GA; a tight deadline (α=1.0) and a max-accuracy bound ($t_{\max} \to \infty$) bracket the range. For the bandwidth experiment we add a bandwidth-blind greedy that places tasks as if transfers were free: it never runs the ℓ-shrinking loop of Algorithm 1, so it reports bound-level scores even when its schedules miss the deadline under real link costs.

TABLE II: Per-dataset test runs on the seeded instances (small: α=1.2; medium, large, xlarge: α=1.5). Score is the average achieved accuracy $\bar{A}$; all listed schedules meet the deadline. "–" means not run: the MILP is capped at $n\,m \le 800$ variables, and rounding is skipped on the two largest datasets. The experiment figures use different seeds and sizes, so their bounds differ from these values.

Dataset	n	m	Method	Score	Makespan	Time (s)
small	10	3	MILP	1.637	32.35	0.004
			Rounding	1.637	32.35	0.17
			Greedy	1.637	30.29	0.000
			GA	1.637	35.93	0.27
medium	100	5	MILP	1.443	119.15	0.39
			Rounding	1.443	118.83	26.59
			Greedy	1.443	82.51	0.003
			GA	1.360	117.20	5.37
large	500	8	Greedy	1.419	101.88	0.04
			GA	1.200	132.65	21.44
xlarge	1000	10	Greedy	1.406	170.41	0.15
			GA	1.128	216.02	73.13

Unless a varied parameter says otherwise, every instance uses the default configuration: n=50 tasks, m=5 fog nodes, slack $\alpha = 1.5$, edge density 0.15, and ten replications per point drawn from two independent seed sets of five, so the reported means do not hinge on a single instance draw. Error bars show one standard deviation; randomized methods are repeated over seeds 0–9. The MILP is solved with a 60 s time limit and skipped above 800 assignment variables ($n\,m$); a time-limited run reports its best incumbent, which can sit below an achievable bound the solve has not certified. Every MILP run that finished below another method on the same instance was therefore re-solved with a 300 s limit and a 10^{-6} optimality gap, and the figures report these certified scores.

Metrics recorded per run: the achieved accuracy score $\bar{A} = \frac{1}{n}\sum_i A_i\,\ell_i$, where $\ell_i = \sum_a l_{ia}$. Because each A_i is an accuracy weight rather than a probability, the score is read relative to the max-accuracy bound of the same instance, drawn dashed in every figure. Makespan and deadline adherence are also recorded.

Table II reports the per-dataset test runs; these use fixed seeded instances that differ from the replication instances behind the experiment figures, so their scores and bounds need not coincide with the experiment means. Two trends are already visible: the greedy and the MILP achieve the same score wherever both run, because under the tested slack all tasks execute fully and placement only moves the makespan; and the GA's score drops as the workflow grows, because its fixed population budget must cover an n-dimensional search space. Note that the MILP maximizes accuracy, not makespan: on the medium dataset it and the greedy reach the same score while their makespans differ (119.15 vs. 82.51), since any placement

that runs all tasks fully and meets the deadline is optimal for the objective.

Reproducibility. The simulator, instances, per-run results, MILP solver logs, and an executed notebook are publicly available at https://github.com/gayanigupta/data-aware-scheduling; the implementation depends only on NumPy, SciPy (HiGHS), and pandas.

B. Deadline Slack

Fig. 3a shows the reported accuracy score against the slack factor α. The methods separate at the tight deadline: at $\alpha = 1.0$ the MILP stays within about 2% of the bound (1.40 vs. 1.43), rounding and the greedy follow at about 1.32, and the GA is lowest at 1.26, about 12% below the bound. From $\alpha = 1.2$ onward the greedy reaches the bound exactly, and after the certification re-solves so does the MILP: the runs that were still improving when the 60 s limit hit all certified the bound within 300 s, confirming that the gap was a truncated incumbent rather than a weaker optimum. The two stochastic methods never fully close the gap: rounding sits about 3% below at $\alpha = 1.2$ and reaches the bound at $\alpha = 1.5$–2.0, while the GA trails by about 1–5% across the range. The scheduling choice matters most when the workflow barely fits its deadline, and a fixed sampling budget does not recover the bound even when one exists.

Pushing α below 1.0 forces partial execution: no schedule can run every task fully, so a method must choose what to sacrifice (Fig. 3d). At $\alpha = 0.5$ the methods spread clearly—MILP holds about 76% of the bound, the GA 71%, rounding 68%, and the greedy 46% (74% for its path-shrinking variant of Section IV-B)—and the ranking holds at 100 tasks. Two limits surface there: the GA returns infeasible schedules at $\alpha \leq 0.8$, and every 100-task MILP solve at $\alpha \leq 0.9$ exhausted the 60 s limit (incumbents, not optima), with rounding and the greedy far cheaper. From $\alpha \approx 1.2$ all feasible methods reach the bound.

C. Federation Size

Fig. 3b shows the reported accuracy score as the federation grows from $N = 2$ to $N = 10$ nodes at the default slack $\alpha = 1.5$. Accuracy is already near the ceiling, so extra nodes do not raise it; what they add is room to schedule. The MILP and the greedy match the bound at every size. The stochastic methods drift downward as the federation widens: the GA goes from about 1% below the bound at N=2 to about 2% below at N=10, while rounding tracks within about 1% of the bound until N=8 and lands about 0.5% short at N=10—their fixed search budgets cover a shrinking fraction of the placement space. Extra capacity does not raise accuracy once the deadline is met; its value is in the feasible placements it opens when the deadline is tight.

D. Workflow Size

Fig. 3c shows accuracy for DAGs of 10, 100, and 1000 tasks (these instances use different seeds than Table II, so the bounds differ). At 10 tasks all four methods sit on the bound (1.44); at 100 the MILP and the greedy match it (1.39), rounding follows at 1.39, and the GA drifts first at 1.31. At 1000 tasks the MILP exceeds our 800-variable cap, the greedy stays on the bound (1.39), rounding follows at 1.39 with its trial count cut to four, and the GA falls to 1.12: its fixed population budget must cover a search space that grows with n. On wall time, at 1000 tasks the greedy answers in about 0.1 s, the GA in roughly 10^2 s, and rounding a few hundred seconds more.

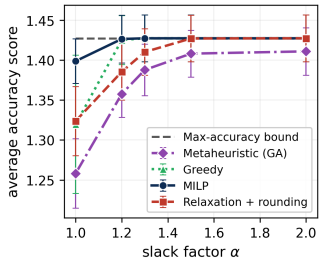
E. Task Failures

We model failures with a retry-once rule: when task T_i fails on its node, it runs once more in place, paying t_{ia} a second time, and it keeps its accuracy if the retry still finishes by the deadline. Fig. 3e shows score and deadline feasibility as the per-task failure rate rises from 0% to 50%. Scores do not move, since retries preserve accuracy; the methods separate on feasibility instead. The greedy is the only method whose schedule stays feasible at every rate: its placements leave enough slack to absorb the retries, while the GA meets the deadline in about 90% of runs, the MILP in about 60%, and rounding in about 50% once failures occur. The curves are nearly flat: at 50 tasks even a 10% rate makes some failure almost certain, so feasibility depends on schedule slack, not the rate itself.

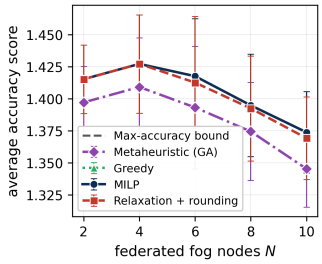
F. Link Bandwidth Degradation

The final experiment stresses the communication cost that motivates the model. On the default instance (n=50, m=5, α=1.5), every inter-node bandwidth B_{ba} is multiplied by a factor from 1.0 down to 0.05 with the deadline fixed, modeling links deteriorating during an event. To show what data-aware placement is worth, we add a bandwidth-blind baseline: the same greedy algorithm as Section IV-B, but choosing placements as if all transfers were free, and then evaluated against the real communication delays.

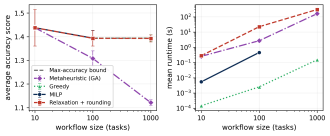
Fig. 3f shows the two metrics that matter. On score alone, the blind baseline looks perfect—it runs every task fully and reports the bound at every level. The right panel shows why that is misleading: the blind schedules miss the deadline already at half bandwidth and are infeasible in every run at 0.1 and below. The data-aware schedulers behave differently. The MILP stays feasible at every level and loses only about 9% of score at 0.05, shrinking execution fractions to fit the deadline. The greedy stays feasible down to 0.1 but pays in accuracy (0.86 at 0.1, mostly infeasible at 0.05). The GA is mostly feasible down to 0.2, survives in only 20% of runs at 0.1, and fails at 0.05. Rounding holds at 0.5 and survives in about 70% of runs at 0.2, but its sampled assignments largely stop passing the feasibility LP below that; this is a limit of our implementation, not a proof that no feasible rounding exists.



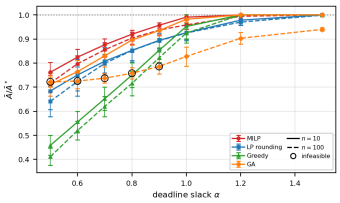
(a) Accuracy vs. deadline slack α.



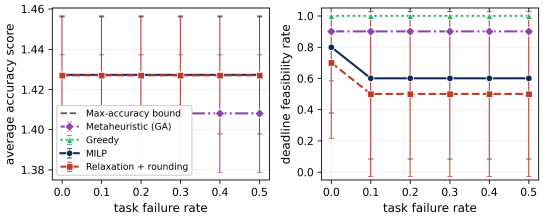
(b) Accuracy vs. federation size N.



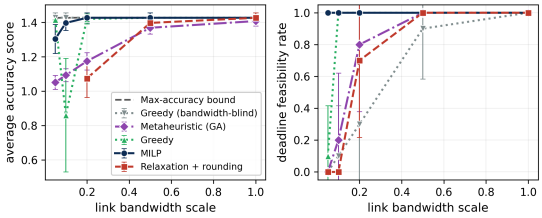
(c) Accuracy vs. workflow size n.



(d) Score normalized by the bound as α drops below 1.0 (10- and 100-task workflows); circled points are infeasible.



(e) Accuracy (left) and deadline feasibility (right) vs. task failure rate.



(f) Accuracy (left) and deadline feasibility (right) as all link bandwidths are scaled down.

Fig. 3: Reported average-accuracy score and deadline feasibility across the five experiments. The methods separate most when the deadline binds (a); extra nodes open placements rather than raising accuracy (b); the greedy tracks the bound at every size in fractions of a second (c); below α=1.0 the methods spread out and the GA loses feasibility at scale (d); retries preserve accuracy, so methods separate on feasibility (e); and the bandwidth-blind baseline reports bound-level scores while missing the deadline below half bandwidth (f).

TABLE III: Baselines on the hardest settings (10-run means). Score is relative to the full-execution bound; deadline met is the feasible-run share.

Setting	Method	Score/bound	Deadline met
α=0.5, n=100	DAG-MEC [5]	1.00	0%
	Greedy (uniform)	0.41	100%
	Greedy (path-shrink)	0.74	100%
	MILP (ours)	0.72	100%
BW scale 0.1	DAG-MEC [5]	1.00	10%
	Blind greedy	1.00	10%
	MILP (ours)	0.98	100%

G. Baseline Comparison

Table III compares against both classes of baselines: the literature method of [5], which counts transfers but runs every task in full, and two ablations of our own schedulers. Each term the model carries fails exactly where the baseline lacking it fails.

H. Discussion

The experiments point to three conclusions. First, the choice of scheduler matters most when the deadline binds, and the separation widens as it tightens: at α=0.5, where partial execution is forced, the MILP holds about 76% of the bound against the greedy's 46%, while by α=1.5 the feasible methods coincide. Second, accuracy under a binding deadline is not free in wall time: at 1000 tasks the greedy finishes in about 0.15 s, rounding spends about 304 s per run, and the GA about 159 s for a markedly worse score. Third, feasibility is the real differentiator under stress: the GA turns infeasible at tight deadlines, and only the data-aware methods survive degraded bandwidth.

Bottlenecks and shortcomings. The evaluation surfaced limits that delimit where the formulation applies. (i) MILP scalability: the model has $n\,N_{\text{fog}}$ binary assignment variables; we cap runs at 800 with a 60 s limit (300 s for certification re-solves), and 18 of the 120 runs reached the limit, all at α=1.0 or under degraded bandwidth—exactly the hard cases. (ii) LP model size: the relaxation introduces one start-time constraint per (task, predecessor, node) triple, about 10^5 rows at n=1000; constraints must be assembled sparsely, a cost rounding pays per trial. (iii) GA under stress: the population does not grow with n, so its score degrades in Fig. 3c, and its penalty-weighted fitness does not enforce feasibility, hence the infeasible plans at $\alpha \leq 0.8$. (iv) Failure semantics: retry-once differentiates schedulers through feasibility, but replication and checkpointing are left out of the model. (v) Dense instances: per-edge data is stored densely, so the 10^6-task structures we generate cannot yet be scheduled; sparse storage would extend the model to large deployments.

VI. Conclusion and Future Work

We set out to answer a practical question: when the cloud is unreachable, how should a federation of nearby fog machines share a deadline-bound workflow whose tasks

can be run partially? This paper gives the problem a cost model that charges real money for moving data between sites, writes it as a mixed-integer linear program, and offers three faster schedulers that all work under the same model.

The experiments tell a consistent story. When the deadline is generous, every reasonable method runs every task fully and the choice of scheduler barely matters. As the deadline tightens below the point where full execution fits, the methods spread apart: the MILP sacrifices the least accuracy, the uniform-fraction greedy the most, and at 100 tasks the GA stops finding feasible schedules at all. Failures and degraded links separate the methods on feasibility rather than score, and the bandwidth-blind baseline shows exactly what goes wrong when placement ignores data movement—perfect-looking accuracy with deadlines missed.

Future work follows the shortcomings we documented: larger workflows need decomposed or sparser formulations, the GA needs a budget that grows with problem size and a feasibility-first fitness, and the failure model should grow beyond retry-once, with validation on a real federated fog testbed.

Acknowledgments

This research is supported by the National Science Foundation (NSF).

Appendix

This appendix shows that the formulation of Section III-D means what it should. A solution is feasible if every task is placed on exactly one permitted node, at most one l_{ia} per task is non-zero, and all tasks complete by $t_{\max}$.

Lemma 1. In any feasible solution: (i) $P_{ia} = 0 \Rightarrow e_{ia} = 0$; (ii) each task is assigned to exactly one node; (iii) at most one l_{ia} per task is non-zero, and $l_{ia} > 0 \Rightarrow e_{ia} = 1$; (iv) the start time M_{ia} respects data arrival, so T_i cannot start on f_a before every predecessor has completed and its output has reached f_a; (v) on the node that runs T_i, $M_{ia} \le y_i$ and $y_i = M_{ia} + t_{ia} l_{ia}$ is its completion time; and (vi) every task completes by $y_i \le t_{\max}$.

Proof. (i) follows from (4), (ii) from (3), and (iii) from (5) with (ii). For (iv), induct on the distance from a source: a source has $M_{ia} = 0$ by (7); for any other task, (6) gives $M_{ia} \ge \max_{j \in \mathrm{in}_i}(y_j + \sum_b e_{jb} c^{ba}_{ji})$, exactly the data-arrival bound. (v) is (8), (vi) is (9). The model only lower-bounds M_{ia}; the earliest-start schedule is a special case. □

Theorem 1. Feasible solutions of (2)–(10) and valid schedules are in one-to-one correspondence, and the optimal value of (2) is the maximum achievable average weighted accuracy.

Proof. Lemma 1 shows a feasible (e, l, M, y) is a valid schedule whose accuracy (2) counts exactly. Conversely, from any valid schedule set $e_{ia} = 1$ on the node running T_i, l_{ia} to the fraction executed, M_{ia} and y_i to the realized times; validity implies (3)–(10) with equal objective value. The two sets correspond, so the MILP optimum equals the best achievable accuracy. □

For an MILP solver, (8) contains the quadratic products $e_{ia} M_{ia}$. With $U = t_{\max} + \max_{j,i,b,a}\{c^{ba}_{ji}\}$, bound

$$M_{ia} \le U, \qquad \forall f_a,\ T_i, \tag{14}$$

introduce $z_{ia} = e_{ia} M_{ia}$, and replace (8) with

$$y_i = \sum_{f_a \in F} z_{ia} + \sum_{f_a \in F} t_{ia} l_{ia}, \qquad \forall T_i, \tag{15}$$

together with the standard big-M constraints

$$0 \le z_{ia} \le e_{ia}\, U, \qquad \forall f_a,\ T_i, \tag{16}$$

$$M_{ia} - (1 - e_{ia})\, U \le z_{ia} \le M_{ia}, \qquad \forall f_a,\ T_i. \tag{17}$$

Theorem 2. The linearized formulation admits exactly the same feasible (e, l) pairs as (2)–(10), and attains the same optimal value of (2).

Proof. No M_{ia} needs to exceed U: every lower bound in (6) is at most $y_j + \max c^{ba}_{ji} \le U$, so larger values can be clamped without touching the objective, and $M_{ia} \ge 0$ holds by (7)–(6). When $e_{ia} = 1$, (17) squeezes $z_{ia} = M_{ia} = e_{ia} M_{ia}$; when $e_{ia} = 0$, (16) forces $z_{ia} = 0 = e_{ia} M_{ia}$. Hence $z_{ia} = e_{ia} M_{ia}$ in all cases, and since $l_{ia} = 0$ whenever $e_{ia} = 0$ by (5), (15) reduces to (8). Every other constraint is shared by both formulations, so their feasible (e, l) sets and optima coincide. □

References

[1] R. F. Hussain and M. A. Salehi, "Resource allocation of Industry 4.0 micro-service applications across serverless fog federation," Future Generation Computer Systems, vol. 154, pp. 479–490, 2024.

[2] C.-H. Hong and B. Varghese, "Resource management in fog/edge computing: A survey on provisioning, infrastructure, and algorithms," ACM Computing Surveys, vol. 52, no. 5, pp. 1–37, 2019.

[3] G. Gupta, A. Gupta, M. A. Salehi, A. Fernández Anta, S. Bhowmick, J. Aguilar, and M. Kourouma, "Towards efficient allocation of tasks in DAG-based workflows across federated fog systems," in 2025 IEEE Cloud Summit, pp. 124–129, 2025.

[4] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," IEEE Transactions on Parallel and Distributed Systems, vol. 13, no. 3, pp. 260–274, 2002.

[5] G. Li, H. Tan, L. Liu, H. Zhou, S. Jiang, Z. Han, X.-Y. Li, and G. Chen, "DAG scheduling in mobile edge computing," ACM Transactions on Sensor Networks, 2023.

[6] S. G. Ahmad, C. S. Liew, M. M. Rafique, E. U. Munir, and S. U. Khan, "Data-intensive workflow optimization based on application task graph partitioning in heterogeneous computing systems," in 2014 IEEE Fourth International Conference on Big Data and Cloud Computing, pp. 129–136, 2014.

[7] F. Faticanti, F. De Pellegrini, D. Siracusa, D. Santoro, and S. Cretti, "Throughput-aware partitioning and placement of applications in fog computing," IEEE Transactions on Network and Service Management, vol. 17, pp. 2436–2450, 2020.

[8] J. Gentry, C. Denninnart, and M. Salehi, "Robust dynamic resource allocation via probabilistic task pruning in heterogeneous computing systems," in Proceedings of IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 375–384, 2019.